

Solution STA510 Exam fall 2023

Problem 1

Consider a random variable X with probability density

$$f(x) = cx(1-x), \quad 0 \leq x \leq 1.$$

a)

For $f(x)$ to be a proper density, we must have that $\int_0^1 f(x)dx = 1$.

$$\begin{aligned} \int_0^1 f(x)dx &= \int_0^1 cx(1-x)dx = c \int_0^1 (x-x^2)dx \\ &= c \cdot \left[\frac{1}{2}x^2 - \frac{1}{3}x^3 \right]_0^1 \\ &= c \left(\frac{1}{2} - \frac{1}{3} - 0 \right) = \frac{1}{6}c \end{aligned}$$

solving $\int_0^1 f(x)dx = 1$ we obtain $c = 6$.

b)

For the mean, we have

$$\begin{aligned} E(X) &= \int_0^1 xf(x)dx = 6 \int_0^1 (x^2 - x^3)dx \\ &= 6 \cdot \left[\frac{1}{3}x^3 - \frac{1}{4}x^4 \right]_0^1 \\ &= 6 \cdot \left(\frac{4}{12} - \frac{3}{12} \right) = 6 \cdot \frac{1}{12} = \frac{1}{2}. \end{aligned}$$

Further, to calculate $Var(X) = E(X^2) - E(X)^2$, we need to find $E(X^2)$ by

$$\begin{aligned} E(X^2) &= \int_0^1 x^2 f(x)dx = 6 \int_0^1 (x^3 - x^4)dx \\ &= 6 \cdot \left[\frac{1}{4}x^4 - \frac{1}{5}x^5 \right]_0^1 \\ &= 6 \cdot \left(\frac{1}{4} - \frac{1}{5} \right) = 6 \cdot \frac{1}{20} = \frac{3}{10}. \end{aligned}$$

The variance is then given by

$$Var(X) = E(X^2) - (E(X))^2 = \frac{3}{10} - \frac{1}{4} = \frac{6}{20} - \frac{5}{20} = \frac{1}{20}.$$

c)

Show that the cumulative distribution function $F(x)$ is given by $F(x) = x^2(3-2x)$, $0 \leq x \leq 1$.

We have

$$F(x) = \int_0^x f(y)dy = 6 \int_0^x (y - y^2)dy = 6 \cdot \left[\frac{1}{2}y^2 - \frac{1}{3}y^3 \right]_0^x = 3x^2 - 2x^3 = x^2(3-2x).$$

Use $F(x)$ to calculate $P(X > 0.5)$ and $P(0.1 < X < 0.5)$.

$$P(X > 0.5) = 1 - F(0.5) = 1 - 0.25(3 - 1) = 1 - 0.5 = 0.5.$$

$$P(0.1 < X < 0.5) = F(0.5) - F(0.1) = 0.5 - 0.01(3 - 0.2) = 0.472.$$

Problem 2

Suppose the lifetimes of an electronic component can be described by a Weibull distribution with parameters α and β .

a)

Show that the CDF $F(x)$ is given by $F(x) = 1 - e^{-\alpha x^\beta}$.

We have from the tablesformulas-sheet that the density of the Weibull distribution is given by

$$f(x) = \alpha\beta x^{\beta-1} e^{-\alpha x^\beta}, \quad x \geq 0.$$

The cdf is then given by

$$F(x) = \int_0^x \alpha\beta y^{\beta-1} e^{-\alpha y^\beta} dy = \int_0^x e^{-\alpha y^\beta} \alpha\beta y^{\beta-1} dy$$

and using integration by substitution ($u = \alpha y^\beta$, $\frac{du}{dy} = \alpha\beta y^{\beta-1}$ $\iff du = \alpha\beta y^{\beta-1} dy$, $u(0) = 0$, $u(x) = \alpha x^\beta$) we have

$$F(x) = \int_0^{\alpha x^\beta} e^{-u} du = [-e^{-u}]_0^{\alpha x^\beta} = -e^{-\alpha x^\beta} - (-1) = 1 - e^{-\alpha x^\beta}.$$

b)

Explain (with words and formulas, not code) how we can generate random variables that are Weibull distributed with parameters α and β using the inverse transform method.

We know that if X is a continuous random variable with cdf $F(x)$ then $U = F(X) \sim U[0, 1]$. Thus, we solve the equation

$$\begin{aligned} 1 - e^{-\alpha X^\beta} = U &\iff -e^{-\alpha X^\beta} = U - 1 \iff e^{-\alpha X^\beta} = 1 - U \\ &\iff -\alpha X^\beta = \log(1 - U) \iff X^\beta = -\frac{1}{\alpha} \log(1 - U) \\ &\iff X = \left(-\frac{1}{\alpha} \log(1 - U) \right)^{1/\beta} \end{aligned}$$

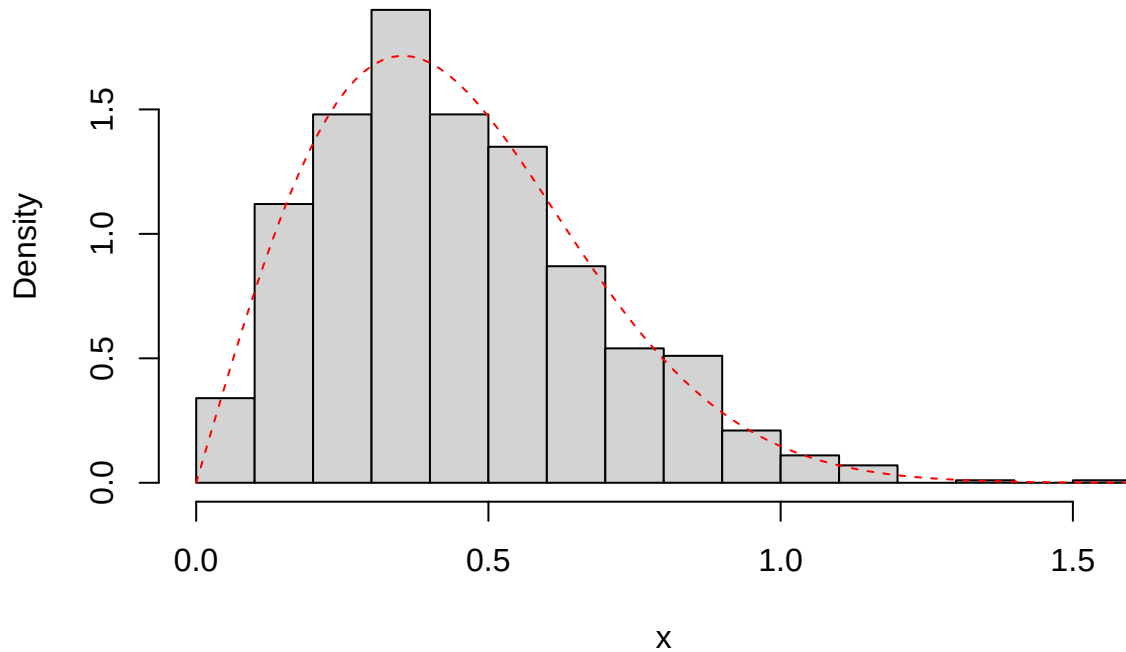
Now, we generate $U_i \sim U[0, 1]$ and get $\left(-\frac{1}{\alpha} \log(1 - U_i) \right)^{1/\beta} \sim \text{Weibull}(\alpha, \beta)$ for $i = 1, \dots, 1000$.

Let's take a look to $X \sim \text{Weibull}(\alpha = 2, \beta = 4)$.

```
# Inverse transformation method
r.myweibull <- function(Nsim,alpha=1,beta=1){
  u <- runif(Nsim,0,1)
  x <- (-(1/alpha)*log(1-u))^(1/beta)
  return(x)
}
set.seed(1)
```

```
alpha=4
beta=2
hist(r.myweibull(1000,alpha,beta), freq=FALSE, xlab="x",
     main=paste0("Histogram of Weibull(alpha=",alpha,"beta=",beta,"))
curve(dweibull(x, shape=beta, scale = (1/alpha)^(1/beta)), col="red" ,lty=2, add=T)
```

Histogram of Weibull(alpha=4,beta=2)



Note we have a different parametrization of the Weibull dist. in R

c)

Let us now assume, we want to estimate the integral $\int_{0.5}^{2.5} f(x)dx$ crude Monte Carlo estimator. Describe the steps with words and formulas.

If $\theta = \int_a^b f(x)dx$ then the crude Monte Carlo estimator is given by

$$\hat{\theta}_{CMC} = \frac{b-a}{n} \sum_{i=1}^n f(U_i) \text{ with } U_i \sim U[a, b].$$

Thus, we need to simulate $n = 10000$ iid uniform random variables $U[0.5, 2.5]$ and to calculate $\hat{\theta}_{CMC} = \frac{2}{n} \sum_{i=1}^n f(X_i)$.

The standard deviation of the crude Monte Carlo estimator $\hat{\theta}_{CMC}$ can be estimated by

$$\hat{\sigma}(\hat{\theta}_{CMC}) = \frac{b-a}{\sqrt{n}} \sqrt{\frac{1}{n-1} \sum (f(U_i) - \overline{f(U)})^2} = \frac{2}{100} \sqrt{\frac{1}{9999} \sum (f(U_i) - \overline{f(U)})^2}.$$

```
f.myweibull <- function(x,alpha=1,beta=1){
  return(alpha*beta*x^(beta-1)*exp(-alpha*x^beta))
}
```

```

Nsim <- 10000
alpha=4
beta=2
x1 <- runif(Nsim,min=0.5,max=2.5)
intcMCvec <- 2*mean(f.myweibull(x1,alpha,beta)) # crude MC result
cat("CMC estimate: ", mean(intcMCvec), "\n", "True value: ",
    pweibull(2.5,shape=beta,scale=(1/alpha^(1/beta)))-
    pweibull(0.5,shape=beta,scale=(1/alpha^(1/beta))), "\n" )

## CMC estimate:  0.3738885
## True value:  0.3678794

```

d)

Now, we want to improve our crude Monte Carlo estimator using antithetic variables. Describe the steps with words and formulas for $n = 10000$ simulations.

First, we recognize from the plot of the *Weibull*(4,2) distribution that our function is monotonic on the interval $[0.5, 2.5]$. Thus, we can improve the crude MC estimator by using pairs of antithetic random variables U_i and V_i with $U_i \sim U[0.5, 2.5]$ and $V_i = 0.5 + 2.5 - U_i = 3 - U_i$. The antithetic MC is then given by

$$\hat{\theta}_{AT} = \frac{b-a}{n} \sum_{i=1}^{n/2} (f(U_i) + f(V_i)) \text{ with } n = 10000.$$

Let us check the performance of the crude MC and antithetic MC estimator.

```

nrep <- 1000
Nsim <- 10000
intcMCvec <- numeric(nrep)
intaMCvec <- numeric(nrep)
for(i in 1:nrep){
  x1 <- runif(Nsim,min=0.5,max=2.5)
  intcMCvec[i] <- 2*mean(f.myweibull(x1,alpha,beta)) # crude MC result
  x2 <- runif(Nsim/2,min=0.5,max=2.5)
  x2 <- c(x2,3-x2)
  intaMCvec[i] <- 2*mean(f.myweibull(x2,alpha,beta)) # antitetic MC result
}
tv <- pweibull(2.5,shape=beta,scale=(1/alpha^(1/beta)))-
    pweibull(0.5,shape=beta,scale=(1/alpha^(1/beta))) # true value
data.frame(estimate = round(c(mean(intcMCvec),mean(intaMCvec),tv),digits=5),
    std = round(c(sd(intcMCvec), sd(intaMCvec),0),digits=5),
    row.names = c("Crude MC","Antithetic MC", "True"))

##           estimate      std
## Crude MC      0.36783 0.00700
## Antithetic MC  0.36789 0.00622
## True          0.36788 0.00000

# Estimated std of CMC using formula
set.seed(1)
x1 <- runif(Nsim,min=0.5,max=2.5)
sdCMC <- (2 / sqrt(Nsim)) * sd(f.myweibull(x1,alpha,beta))
sdCMC

## [1] 0.007306701

```

Do you think standard deviation of antithetic MC estimator will more or less compared to the standard deviation of crude MC estimator? Justify your answer. Answer: We can expect a lower standard deviation for the antithetic MC estimator compared to the crude MC estimator because we have constructed antithetic pairs (U_i, V_i) and $f(x)$ with $Cov(f(U_i), f(V_i)) < 0$ and $f(x)$ is monotonic on the interval $[0.5, 2.5]$.

e)

From 2b), we know how to generate random variables that are Weibull distributed with parameters α and β using the inverse transform method, namely by $(-\frac{1}{\alpha}\log(1 - U_i))^{1/\beta} \sim Weibull(\alpha, \beta)$. Further, we have $X_3 \sim Exp(\alpha)$ (in rate parametrization as $E(X_3) = \frac{1}{\alpha}$). Using the tablesformulas-sheet, we see that Exponential distribution is a special case of the Weibull distribution with $\beta = 1$. Thus, $X_3 \sim Weibull(\alpha, 1)$. To generate $n = 10000$ random samples for S , we can repeat the following 10000 times:

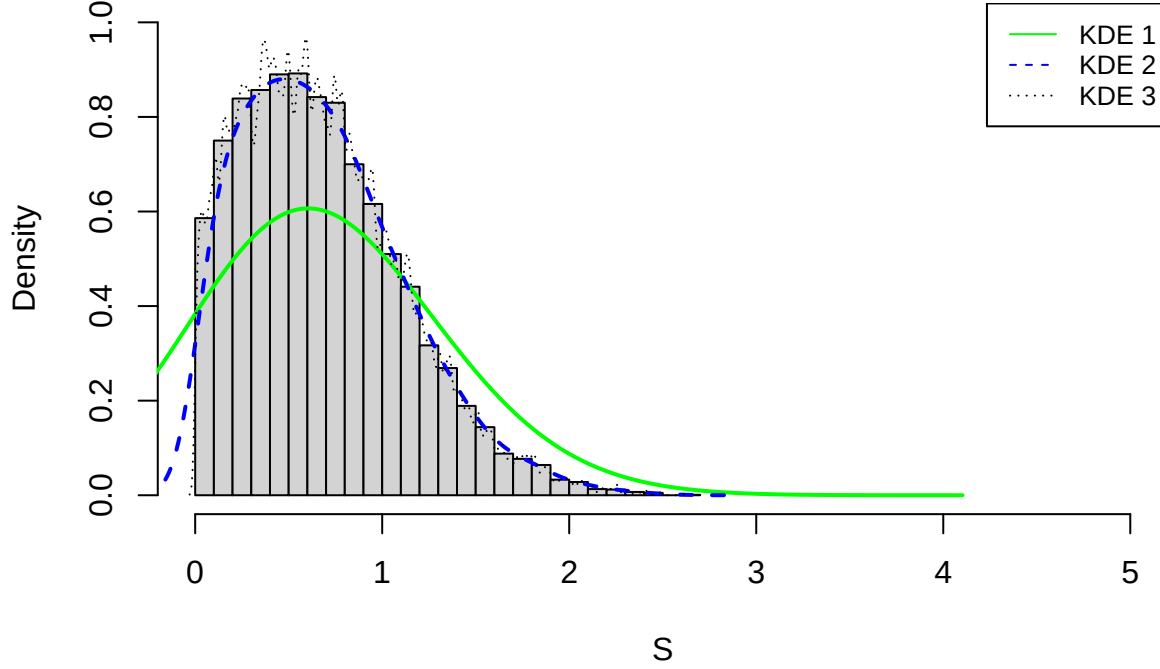
1. $X_1 = (-\frac{1}{\alpha}\log(1 - U))^{1/\beta}$ with $U \sim U[0, 1]$
2. $X_2 = (-\frac{1}{\alpha}\log(1 - U))^{1/\beta}$ with $U \sim U[0, 1]$
3. $X_3 = -\frac{1}{\alpha}\log(1 - U)$ with $U \sim U[0, 1]$
4. $S = \min(X_1, X_2, X_3)$

```
r.lifetime <- function(Nsim,alpha=1,beta=1){
  X <- - (1/alpha) *log(1-runif(Nsim))
  Y <- (- (1/alpha)*log(1-runif(Nsim)))^(1/beta)
  Z <- (- (1/alpha)*log(1-runif(Nsim)))^(1/beta)
  return(pmin(X,Y,Z))
}

alpha <- 0.5
beta <- 2

set.seed(1)
S <- r.lifetime(10000,alpha=alpha,beta=beta)
hist(S, breaks=20, freq=F, xlim=c(0,5), ylim=c(0,1),
     main="Histogram of S and KDE")
lines(density(S,bw=0.5), lwd=2, lty=1, col="green", xlim=c(0,60))
lines(density(S,bw=0.1), lwd=2, lty=2, col="blue", xlim=c(0,60))
lines(density(S,bw=0.01),lwd=1, lty=3, col="black", xlim=c(0,60))
legend("topright", legend=c("KDE 1", "KDE 2", "KDE 3"),
     col=c("green", "blue","black"), lty = 1:3, cex=0.8)
```

Histogram of S and KDE



f)

What are the pros and cons for a histogram versus a kernel density estimate (KDE)? Both, the histogram and the KDE are ways to estimate the underlying unknown probability density function. And both are useful for presenting results. The histogram requires a selection of the number of bins whereas for KDE we need to select the bandwidth. Both, the number of bins and the bandwidths influence the level of smoothing. Depending on the number of bins that shape of histogram can change quite dramatically. On the other hand, histograms are easy to interpret. KDEs are in particular suited for smooth and continuous distributions but they can be computationally intensive.

What is the difference between the three different KDEs in the figure? The different lines correspond to different bandwidths that influence the level of smoothing. For KDE 1 (solid line), we have a large bandwidth and observe an oversmoothing. For KDE 2 (dashed line), we have a reasonable bandwidth, i.e. a good trade-off of smoothing and low bias. For KDE 3 (dotted line), we have a very small bandwidth, resulting in a very wiggly line, i.e. we observe a high variance.

Which of the three KDEs is best? Most justify your answer. KDE 2 looks best under these three options. It shows a good bias-variance trade-off. In general, a small bandwidth leads to low bias but high variance and a large bandwidth leads to low variance but high bias. We aim to find some optimal trade-off between variance and bias, e.g., by the mean integrated square error.

g)

Suppose the parameter $\beta = \frac{3}{2}$ is given and only the parameter $\theta = \alpha$ is unknown for $X_1, X_2 \sim \text{Weibull}(\theta, \frac{3}{2})$ and $X_3 \sim \text{Exp}(\theta)$. It can be shown that the density function of $S = \min(X_1, X_2, X_3)$ has the form

$$f_S(x) = \begin{cases} \theta(1 + 3x^{1/2})e^{-\theta(x+2x^{3/2})}, & \text{if } x \geq 0, \\ 0, & \text{if } x < 0. \end{cases}$$

Further, let us assume, we observe $n = 20$ lifetimes x_i for the sequential system (in years)

0.62, 0.61, 1.64, 0.42, 0.45, 0.98, 0.09, 0.98, 1.91, 0.13
 0.46, 0.39, 1.10, 0.55, 0.28, 0.35, 0.83, 0.37, 0.96, 1.03

We want to estimate the parameter θ by a Bayesian approach using $\theta \sim \text{Gamma}(a = 15, b = \frac{1}{20})$ as a prior. Thus,

$$E(\theta) = a \cdot b = \frac{15}{20} = \frac{3}{4} = 0.75$$

$$\text{Var}(\theta) = a \cdot b^2 = \frac{15}{20^2} = 0.0375$$

h)

Given the two constants $c_1 = \prod_{i=1}^n (1 + 3x_i^{1/2})$ and $c_2 = \sum_{i=1}^n (x_i + 2x_i^{3/2})$, show that the the Likelihood function can be written as $L(\theta) = \theta^n c_1 e^{-\theta c_2}$.

Given $\beta = \frac{3}{2}$, it can be shown that the density function of $S = \min(X_1, X_2, X_3)$ has the form

$$f_S(x) = \begin{cases} \theta(1 + 3x^{1/2})e^{-\theta(x+2x^{3/2})}, & \text{if } x \geq 0, \\ 0, & \text{if } x < 0. \end{cases}$$

The Likelihood function given the observation $x_1, \dots, x_n$ is

$$L(\theta) = \prod_{i=1}^n f_S(x_i) = \theta^n \prod_{i=1}^n (1 + 3x_i^{1/2}) e^{-\theta \sum_{i=1}^n (x_i + 2x_i^{3/2})} = \theta^n c_1 e^{-\theta c_2}$$

for $\theta \geq 0$ and $c_1 = \prod_{i=1}^n (1 + 3x_i^{1/2})$ and $c_2 = \sum_{i=1}^n (x_i + 2x_i^{3/2})$.

i)

In order to simulate from the posterior distribution $p(\theta|x)$, we want to implement a random walk metropolis hasting algorithm. For this, we need to specify the log-target density kernel which is proportional to the posterior distribution. Which of the four options are valid log-target density kernel. Justify your answer.

We have $p(\theta|x) \propto L(\theta)p(\theta)$.

```
# Observed life times for S
LT <- c(0.62, 0.61, 1.64, 0.42, 0.45,
        0.98, 0.09, 0.98, 1.91, 0.13,
        0.46, 0.39, 1.10, 0.55, 0.28,
        0.35, 0.83, 0.37, 0.96, 1.03)

# Definition of density function for S
f.lifeTime <- function(x,theta=1){
  fx <- theta*(1+(3*x^(1/2)))*exp(-theta*(x+2*x^(3/2)))
  return(fx)
}

# Log-target density kernels for the posterior
a = 15
b = 1/20
```

```
# Option 1
lp_temp1 <- function(theta){
  log.like <- log(prod(f.lifeTime(LT,theta=theta)))
  log.prior <- dgamma(theta,shape=a,scale=b,log=TRUE)
  return(log.like+log.prior)
}
```

Option 1 is a valid log-target density kernel because $\log(L(\theta)p(\theta)) = \log(L(\theta)) + \log(p(\theta))$ and $\log(L(\theta)) = \log(\prod_{i=1}^n f_S(x_i))$. Further, $p(\theta) = \frac{1}{b^a \Gamma(a)} x^{a-1} e^{-\frac{1}{b}x}$ is Gamma density as $\theta \sim \text{Gamma}(a, b)$.

```
n <- length(LT)
logc1 <- sum(log(1 + 3*LT^{1/2}))
c2 <- sum(LT + 2*LT^{3/2})
```

```
# Option 2
lp_temp2 <- function(theta){
  log.like <- logc1 + n*log(theta) - c2*theta
  log.prior <- dgamma(theta,shape=a,scale=b,log=TRUE)
  return(log.like+log.prior)
}
```

Option~2 is a valid log-target density kernel. In 2h) we showed that $L(\theta) = \theta^n c_1 e^{-\theta c_2}$ with $c_1 = \prod_{i=1}^n (1 + 3x_i^{1/2})$ and $c_2 = \sum_{i=1}^n (x_i + 2x_i^{3/2})$. Thus, we can write $\log(L(\theta)) = n * \log(\theta) + \log(c_1) - \theta c_2$ with $\log(c_1) = \log(\prod_{i=1}^n (1 + 3x_i^{1/2})) = \sum_{i=1}^n (\log(1 + 3x_i^{1/2}))$. Further, for $\log(p(\theta))$ we have the same as in option 1.

```
# Option 3
lp_temp3 <- function(theta){
  log.like <- logc1 + log(theta^n) - c2*theta # log-likelihood
  log.prior <- log(theta^(a-1)) - log(theta/b) # log-prior
  return(log.like+log.prior)
}
```

Option~3 is not a valid log-target density kernel. The log-Likelihood is correct because the expression is identical to Option~2, because $n * \log(\theta) = \log(\theta^n)$. However, the log prior is not correct. We have

$$p(\theta) = \frac{1}{b^a \Gamma(a)} x^{a-1} e^{-\frac{1}{b}x} \propto x^{a-1} e^{-\frac{1}{b}x}$$

and thus $\log(p(\theta)) \propto \log(\theta^{a-1}) - \frac{\theta}{b}$ and $\frac{\theta}{b} \neq \log(\frac{\theta}{b})$. Finally, Option~4 is a valid log-target density kernel because $\log(p(\theta)) \propto (a-1) \log(\theta) - \frac{\theta}{b}$.

```
# Option 4
lp_temp4 <- function(theta){
  log.like <- logc1 + log(theta^n) - c2*theta # log-likelihood
  log.prior <- (a-1)*log(theta) - theta/b # log-prior
  return(log.like+log.prior)
}
```

j)

```
# general 1d Gaussian proposal random walk MH
oneD.RWMH <- function(lprob, #notice log-density kernel!
  sigma=1.0,
  theta1=0.0,
```



```

                                n.iter=10000){
# space for output
output <- numeric(n.iter)
# first iterate given
output[1] <- theta1
Nacc <- 0
# main iteration loop
for(t in 2:n.iter){
  thetaStar <- rnorm(1,mean=output[t-1],sd=sigma)
  # thetaStar needs to be larger then 0 otherwise p(theta)=0
  # and thus alpha=0, i.e., we reject.
  if(thetaStar > 0){
    alpha <- exp(min(0.0,lprob(thetaStar)-lprob(output[t-1])))
  } else {
    alpha <- 0
  }
  # accept/reject step
  if(runif(1)<alpha && is.finite(alpha)){
    output[t] <- thetaStar
    Nacc <- Nacc + 1
  } else {
    output[t] <- output[t-1]
  }
}
print(paste0("RWMH done, accept rate :",Nacc/(n.iter-1)))
return(output)
}

out1 <- oneD.RWMH(lp_prob=lp_temp4,theta1 = a*b,sigma=0.01,n.iter = 50000)

## [1] "RWMH done, accept rate :0.966839336786736"

out2 <- oneD.RWMH(lp_prob=lp_temp4,theta1 = a*b,sigma=0.5,n.iter = 50000)

## [1] "RWMH done, accept rate :0.229404588091762"

out3 <- oneD.RWMH(lp_prob=lp_temp4,theta1 = a*b,sigma=1.5,n.iter = 50000)

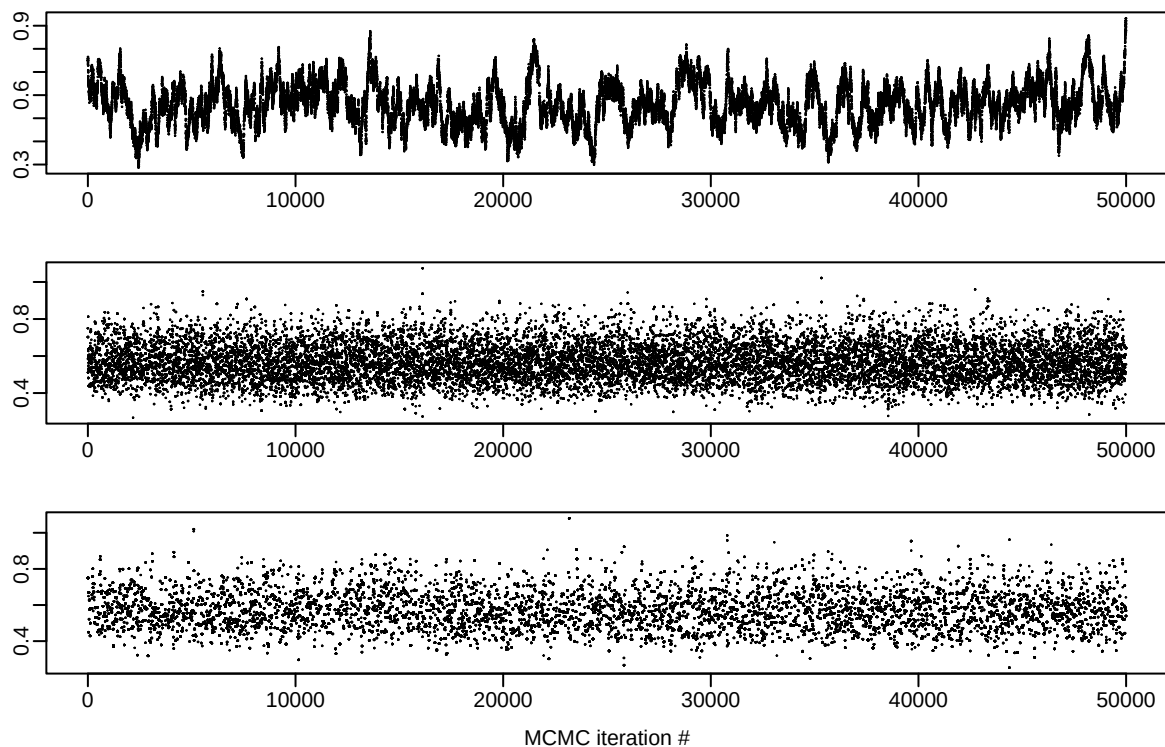
## [1] "RWMH done, accept rate :0.0802416048320966"

par(mfrow=c(3,1),mar=c(3.0, 2.5, 0.5, 0.5),oma=c(3.5,1,1,1),
    mgp=c(2.0,.5,0))
plot(1:length(out1),out1,pch=20,cex=0.1,xlab="",ylab="")
plot(1:length(out2),out2,pch=20,cex=0.1,xlab="",ylab="")
plot(1:length(out2),out3,pch=20,cex=0.1,xlab="MCMC iteration #",ylab="")

library(coda) # mcmc utilites

## Warning: package 'coda' was built under R version 4.1.3

```



```
effectiveSize(out1)
```

```
##      var1
## 144.8566
```

```
effectiveSize(out2)
```

```
##      var1
## 7792.665
```

```
effectiveSize(out3)
```

```
##      var1
## 2587.605
```

Write down a proposal distribution that could be used within the RWMH algorithm. Answer: A common proposal distribution in the RWMH algorithm is the normal distribution $N(\theta_{t-1}, \sigma^2)$ if we have a 1-dim target distribution.

What is the difference between these plots? Answer: The difference between these plots is a different choice of σ^2 . In the top plot, a small σ^2 is chosen, resulting in a process that moves very slowly around the support. In middle plot, we see a good choice of σ^2 , the proposal are quiet spread around the support and it looks also we have a high acceptance rate (dense numbers of dots). The bottom plot indicates a large σ^2 . We see a fewer number of dots which means we have a low acceptance probability because almost all proposal are inconsistent with the target distribution.

How can we justify which of the MCMCs has a good performance? Answer: First, we could check the acceptance rate. For RWMH, we should choose σ^2 so that the accept rate is between 20% – 40%. Further, the Effective Sample Size can used to find a good choice of σ^2 . Diagnostics like Geweke test and Gelman-Rubin test can also be used to check the performance of the MCMC.